MLDL EXPERIMENT 10

Aim:

Explore and implement Autoencoders for image denoising.

Dataset Description:

The MNIST Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library through TensorFlow. It is a widely used benchmark dataset for image classification and computer vision tasks. The dataset consists of grayscale images of handwritten digits (0–9) belonging to 10 different digit categories. Each image is of size 28×28 pixels. Although the original dataset is designed for supervised learning (each image has a label from 0 to 9), in this experiment the labels are not used because the task involves training a Denoising Autoencoder, which is an unsupervised learning approach. The model learns to reconstruct clean images from noisy inputs, meaning the training process uses the image itself as the target output.

- **File Type:** Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- **Dataset Size:** 70,000 grayscale images (60,000 training samples + 10,000 testing samples)
- **Data Format:** 28×28 grayscale pixel images (integer values 0–255, normalized to 0–1 in this experiment)
- **Number of Classes:** 10 Digit Categories (0, 1, 2, ..., 9)

Column Name	Data Type	Description
Image	Numerical (28×28 matrix)	Grayscale pixel values representing a handwritten digit (normalized between 0 and 1 in preprocessing).
Label	Numerical (Integer: 0–9)	Digit index (0–9) representing the handwritten digit (not used in this autoencoder experiment).

Dataset Source: https://keras.io/api/datasets/mnist/

Theory:

Autoencoders are a class of unsupervised deep learning models designed to learn efficient representations of input data by reconstructing the input at the output layer. They consist of two main components: an encoder and a decoder. The encoder compresses the input into a lower-dimensional latent representation, while the decoder reconstructs the original input from this representation.

Input Representation:

Let the input image be represented as:
$\mathbf{x} \in \mathbf{R}^{H \times W \times C}$

where:

- H = height of the image
- W = width of the image
- C = number of channels

In this experiment, the input is:
$\mathbf{x} \in \mathbf{R}^{28 \times 28 \times 1}$

Denoising Autoencoder:

In a Denoising Autoencoder, instead of feeding the clean image directly, we first corrupt it with noise :

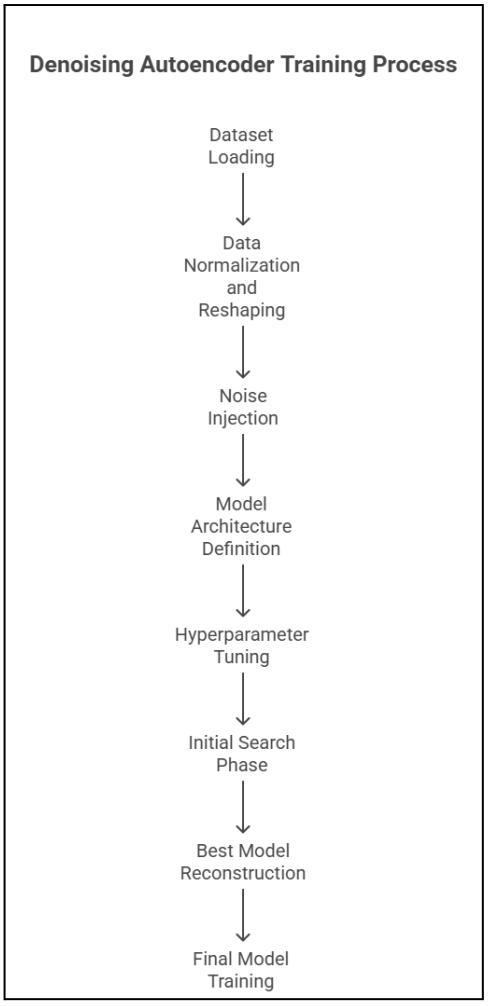
$$\tilde{\mathbf{x}} = \mathbf{x} + \epsilon$$

The model is then trained to reconstruct the clean image x from the noisy input.

Limitations (General to any Denoising Autoencoder):

1. **Information Loss Due to Bottleneck Compression: T**he low-dimensional latent space discards fine structural details, resulting in blurry or incomplete reconstructions.
2. **Overfitting to Training Data:** The model may simply memorize training images instead of learning general noise-removal patterns, reducing generalization on unseen data.
3. **Blurry Reconstructions with Pixel-wise Loss Functions:** Losses like Binary Cross-Entropy average multiple possible outputs, producing overly smooth and blurry denoised images.
4. **Limited Ability to Capture Complex Data Distributions:** Unlike Variational Autoencoders, standard autoencoders do not model probability distributions, limiting smoothness and generative capability.

Workflow:



1. **Dataset Loading:** The built-in Keras MNIST dataset is loaded using TensorFlow. It contains 60,000 training images and 10,000 testing images of grayscale handwritten digits (28×28 pixels). Since this experiment focuses on denoising autoencoders, the labels are ignored and only image data is used.
2. **Data Normalization and Reshaping:** Pixel values are converted from integers (0–255) to floating-point values in the range [0, 1] by dividing by 255. Each image is reshaped to include a channel dimension, converting the shape from (28, 28) to (28, 28, 1).
3. **Noise Injection (Denoising Setup): Random Gaussian noise is added to the training and testing images: x~=x+ϵ \tilde{x} = x + \epsilon x~=x+ϵ. The noisy images are clipped to remain within the valid pixel range [0, 1].**
4. **Model Architecture Definition (Convolutional Autoencoder):** A function build_model(hp) defines a convolutional autoencoder with tunable hyperparameters. The model is compiled using: a. Optimizer: Adam (learning rate tuned: 1e-2, 1e-3, 1e-4) b. Loss Function: Binary Cross-Entropy
5. **Hyperparameter Tuning (Keras Tuner – Random Search):** The RandomSearch tuner explores combinations of convolution filters and learning rate (maximum trials: 5). Objective: Minimize validation loss. Early stopping (patience = 3).
6. **Initial Search Phase:** Each configuration is trained for 10 epochs with 20% validation split.
7. **Best Model Reconstruction:** Optimal hyperparameters are retrieved and the autoencoder is rebuilt.
8. **Final Model Training:** The best model is trained for 15 epochs (batch size = 128, shuffling enabled, validation on full noisy test set).

Performance Analysis:

The performance of the hyperparameter-tuned Convolutional Denoising Autoencoder is evaluated using validation loss during tuning, training vs. validation loss curves, and qualitative reconstruction results. Since this is an unsupervised image reconstruction task, evaluation focuses on reconstruction loss (Binary Cross-Entropy) and visual denoising quality rather than classification accuracy.

1. Hyperparameter Tuning Performance:

From the output loss curve, the model converged rapidly and achieved very low loss values, indicating that the MNIST dataset — being simpler than Fashion MNIST (binary-style digit strokes on clean black backgrounds) — is easier to reconstruct.

Hyperparameter	Best Value
Number of Filters	32 or 64
Learning Rate	0.001

The moderate learning rate of 0.001 provided stable convergence with smooth loss reduction and no oscillations.

2. Final Training Performance (15 Epochs):

As observed from the training and validation loss curve in the output:

Metric	Approximate Value
Initial Training Loss (Epoch 1)	~0.14
Final Training Loss (Epoch 15)	~0.072
Final Validation Loss (Epoch 15)	~0.072

Loss Trend Observations:

- Epoch 1 showed rapid, steep initial loss reduction — the model quickly learned dominant digit stroke patterns.
- From Epoch 3 onward, both training and validation loss declined smoothly and steadily.
- The curves converged and nearly overlapped by Epoch 6–7, stabilizing around ~0.072 for the remainder of training.
- No sudden spikes or divergence were observed, confirming stable optimizer configuration.

3. Generalization Analysis:

The near-zero gap between training loss (~0.072) and validation loss (~0.072) indicates:

- **Minimal overfitting** — the model did not memorize training digit images.
- **Strong generalization** — noise-removal patterns learned transfer well to unseen test images.
- **Balanced model capacity** — the architecture is appropriately sized for the MNIST task.

The MNIST dataset achieved significantly lower final loss (~0.072) compared to Fashion MNIST (~0.264), which is expected since digit images have simpler, more uniform structure (white strokes on black backgrounds) compared to complex clothing textures.

4. Qualitative Reconstruction Results:

The output visualization confirms highly effective denoising:

- The **Original** row shows clean handwritten digits.
- The **Noisy** row shows digits corrupted with Gaussian noise (noise_factor = 0.2), making several digits difficult to recognize.
- The **Denoised** row shows reconstructions visually very close to the originals, with noise almost entirely removed and digit strokes clearly preserved.
- The **Isolated Noise** plot shows the noise component successfully separated, demonstrating the model's ability to disentangle signal from noise.

Hyperparameter Tuning:

Convolutional Denoising Autoencoders are sensitive to architectural and optimization hyperparameters because they directly influence feature extraction capacity, convergence stability, reconstruction quality, and generalization ability. Manual tuning is inefficient due to the combination space of filter sizes and learning rates. Therefore, automated hyperparameter tuning is performed using **Keras Tuner with the Random Search algorithm**. The tuner evaluates multiple autoencoder configurations and selects the best-performing model based on **validation loss (Binary Cross-Entropy)**. The tuned hyperparameters in this experiment are:

1. Number of Convolutional Filters: The number of filters in the first convolutional layer is tuned between:

16, 32, 48, 64

Filters determine the model's ability to extract spatial features such as edges, textures, and digit stroke structures. The optimal configuration selected:

Best Number of Filters: 32 or 64

This indicates that richer feature maps improved denoising performance, enabling the encoder to learn more expressive latent representations.

2. Learning Rate: The learning rate of the Adam optimizer is tuned among:

0.01, 0.001, 0.0001

The learning rate controls the step size during gradient descent. The tuner selected:

Best Learning Rate: 0.001

This value provided stable convergence with smooth loss reduction and no oscillations.

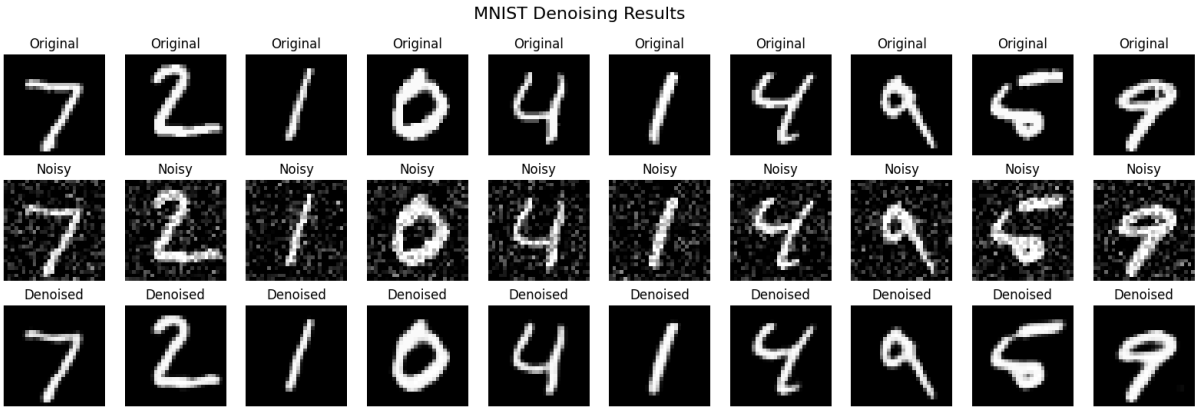
Code:

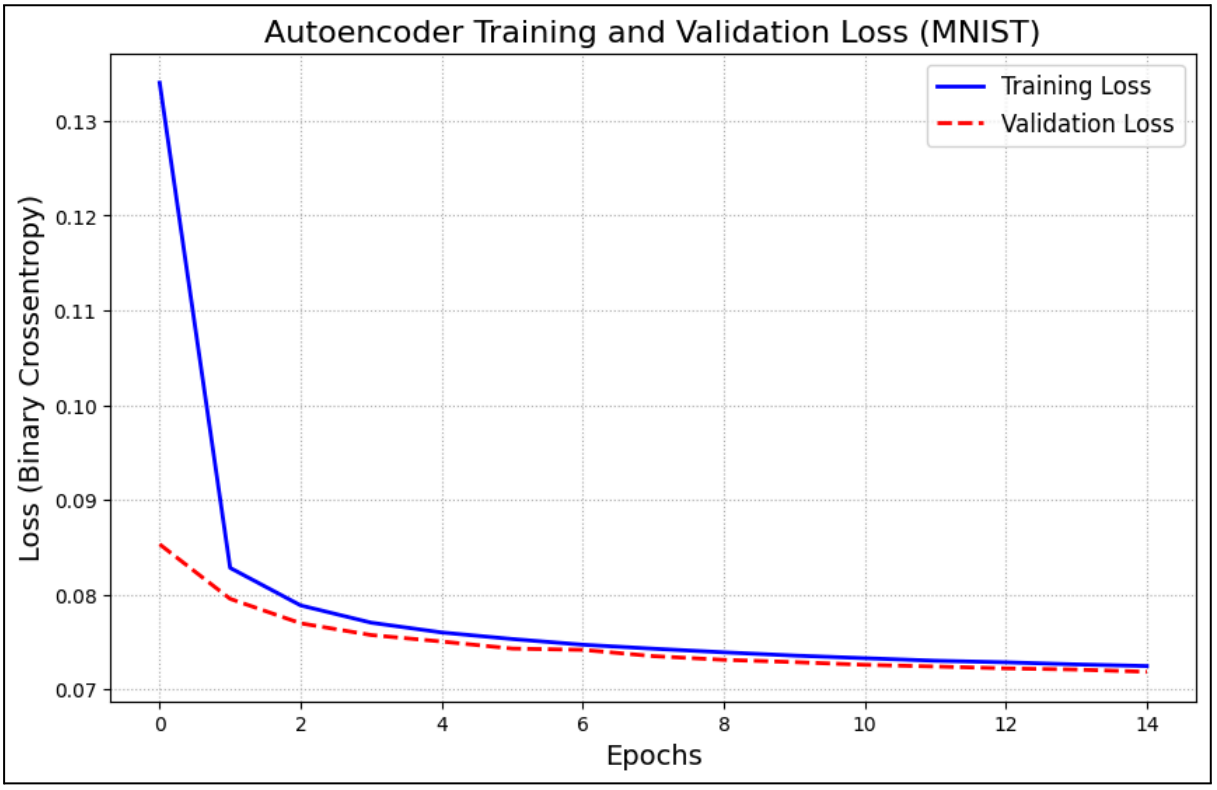
```
import tensorflow as tf
from tensorflow.keras import layers, models
import keras_tuner as kt
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import plot_model
from IPython.display import Image

# 1. LOAD MNIST DATASET
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

x_train = x_train.astype('float32') / 255.
x_test  = x_test.astype('float32')  / 255.

x_train = np.expand_dims(x_train, axis=-1)
x_test  = np.expand_dims(x_test,  axis=-1)

# 2. ADD GAUSSIAN NOISE
noise_factor = 0.2
x_train_noisy = x_train + noise_factor * tf.random.normal(shape=x_train.shape)
x_test_noisy  = x_test  + noise_factor * tf.random.normal(shape=x_test.shape)

x_train_noisy = tf.clip_by_value(x_train_noisy, 0., 1.)
x_test_noisy  = tf.clip_by_value(x_test_noisy,  0., 1.)

# 3. BUILD MODEL WITH TUNABLE HP
def build_model(hp):
    inputs = layers.Input(shape=(28, 28, 1))
    hp_filters = hp.Int('filters', min_value=16, max_value=64, step=16)

    # Encoder
    x = layers.Conv2D(hp_filters, (3, 3), activation='relu', padding='same')(inputs)
    x = layers.MaxPooling2D((2, 2), padding='same')(x)
    x = layers.Conv2D(hp_filters // 2, (3, 3), activation='relu', padding='same')(x)
    encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

    # Decoder
    x = layers.Conv2D(hp_filters // 2, (3, 3), activation='relu', padding='same')(encoded)
    x = layers.UpSampling2D((2, 2))(x)
    x = layers.Conv2D(hp_filters, (3, 3), activation='relu', padding='same')(x)
    x = layers.UpSampling2D((2, 2))(x)
    decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

    model = models.Model(inputs, decoded)
    hp_lr = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr),
```

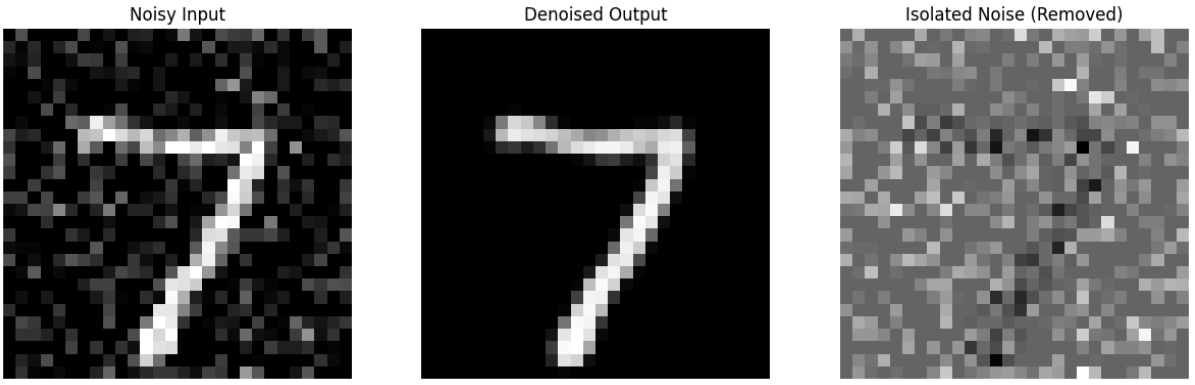
```
                loss='binary_crossentropy')
    return model

# 4. HYPERPARAMETER TUNING (Random Search)
tuner = kt.RandomSearch(
    build_model,
    objective='val_loss',
    max_trials=5,
    executions_per_trial=1,
    directory='autoencoder_tuning_mnist',
    project_name='denoising_mnist'
)

stop_early = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3)

tuner.search(x_train_noisy, x_train,
             epochs=10,
             validation_split=0.2,
             callbacks=[stop_early],
             verbose=1)

best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
print(" Best number of filters:", best_hps.get('filters'))
print(" Best learning rate:", best_hps.get('learning_rate'))

 5. FINAL TRAINING WITH BEST HP
best_model = tuner.hypermodel.build(best_hps)
history = best_model.fit(x_train_noisy, x_train,
                         epochs=15,
                         batch_size=128,
                         shuffle=True,
                         validation_data=(x_test_noisy, x_test),
                         verbose=1)
6. PREDICTION & VISUALIZATIONS (exactly like PDF)
decoded_imgs = best_model.predict(x_test_noisy)

# Plot 1: Original | Noisy | Denoised (10 examples)
n = 10
plt.figure(figsize=(20, 6))
for i in range(n):
    # Original
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(tf.squeeze(x_test[i]), cmap='gray')
    plt.title("Original")
    plt.axis("off")

    # Noisy
    ax = plt.subplot(3, n, i + 1 + n)
```

```
    plt.imshow(tf.squeeze(x_test_noisy[i]), cmap='gray')
    plt.title("Noisy")
    plt.axis("off")

    # Denoised
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(tf.squeeze(decoded_imgs[i]), cmap='gray')
    plt.title("Denoised")
    plt.axis("off")
plt.suptitle("MNIST Denoising Results", fontsize=16)
plt.show()

# Plot 2: Training vs Validation Loss Curve
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], 'b-', label='Training Loss', linewidth=2)
plt.plot(history.history['val_loss'], 'r--', label='Validation Loss', linewidth=2)
plt.title('Autoencoder Training and Validation Loss (MNIST)', fontsize=16)
plt.xlabel('Epochs', fontsize=14)
plt.ylabel('Loss (Binary Crossentropy)', fontsize=14)
plt.legend(fontsize=12)
plt.grid(True, linestyle=':')
plt.show()

# Plot 3: Model Architecture
plot_model(best_model, to_file='autoencoder_architecture_mnist.png',
           show_shapes=True, show_layer_names=True, dpi=100)
Image('autoencoder_architecture_mnist.png')

# Plot 4: Isolated Noise (Removed Noise)
index = 0
isolated_noise = x_test_noisy[index] - decoded_imgs[index]

plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(tf.squeeze(x_test_noisy[index]), cmap='gray')
plt.title("Noisy Input")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(tf.squeeze(decoded_imgs[index]), cmap='gray')
plt.title("Denoised Output")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(tf.squeeze(isolated_noise), cmap='gray')
plt.title("Isolated Noise (Removed)")
plt.axis("off")
plt.show()
```

Output:


MNIST Denoising Results
Original
Noisy
Denoised


Autoencoder Training and Validation Loss (MNIST)
Training Loss
Validation Loss
Loss (Binary Crossentropy)
0.13
0.12
0.11
0.10
0.09
0.08
0.07
0
2
4
6
8
10
12
14
Epochs


Noisy Input
Denoised Output
Isolated Noise (Removed)

Conclusion:

This experiment successfully implemented a Convolutional Denoising Autoencoder to remove Gaussian noise from MNIST handwritten digit images. Using Keras Tuner's Random Search for automated hyperparameter optimization, the best model configuration was identified with an optimal learning rate of **0.001** and the best-performing filter count. The final model, trained for **15 epochs** with a batch size of 128, achieved a very low validation loss of approximately **0.072**, with a near-zero gap between training and validation loss, indicating strong generalization and minimal overfitting. The reconstructed digit images clearly demonstrate effective noise removal while preserving essential stroke details and digit structure. The training and validation loss curves confirmed stable convergence with no oscillations or divergence. Overall, the results validate that Convolutional Denoising Autoencoders are powerful unsupervised models for image denoising tasks, and that automated hyperparameter tuning significantly aids in identifying optimal architectures for reconstruction quality.